Multiagent Systems: an Introduction, Part 3 bis

Viviana Mascardi

viviana.mascardi@unige.it

Dibris

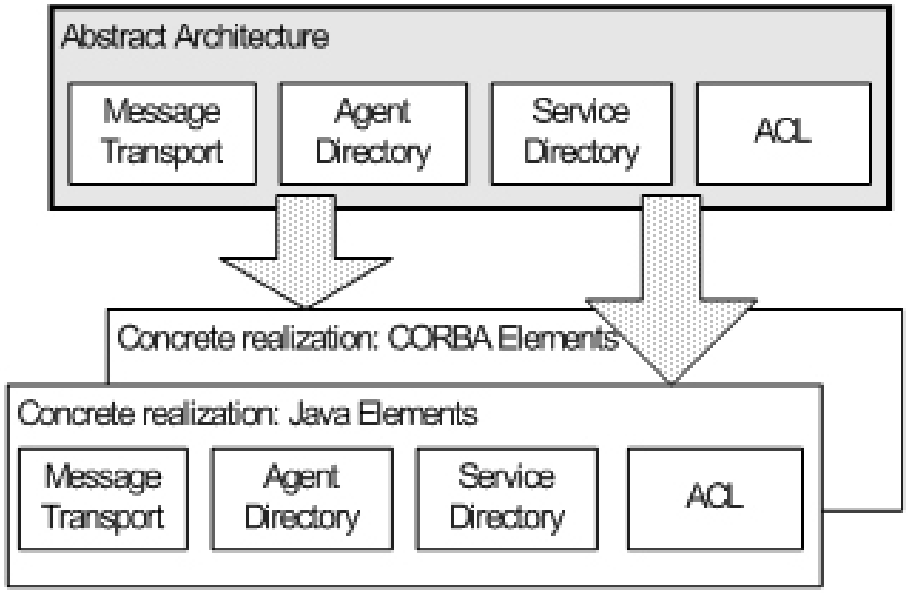
MAS infrastructures

- To support the development and execution of MASs, tools and software infrastructures are needed.
- During the years, various tools have been proposed to transform standard MAS specifications (i.e., AUML specifications) into actual agent code, and a variety of middleware infrastructures have been deployed to provide proper services supporting the execution of distributed MASs.
- In order to be interoperable, such infrastructures should be compliant with the FIPA abstract architecture.

Infrastructures ad-hoc for MASs

FIPA abstract architecture:

`http://www.fipa.org/specs/fipa00001/SC00001L.html`.



JADE

- JADE is one of the most well-known and widely used FIPA-compliant architectures.
- Fabio Luigi Bellifemine, Giovanni Caire, Dominic Greenwood. Developing Multi-Agent Systems with JADE. ISBN: 978-0-470-05747-6, February 2007, Wiley.
- The JADE home page: `http://jade.tilab.com/`
- Latest new relase, December 2022

Dibris

JADE

- JADE (Java Agent DEvelopment Framework) is a software framework fully implemented in Java language.
- It simplifies the implementation of MASs through a middleware that complies with the FIPA specifications and through a set of tools that supports the debugging and deployment phases.
- The agent platform can be distributed across machines (which do not even need to share the same OS) and the configuration can be controlled via a remote GUI.
- The configuration can be changed at run-time by moving agents from one machine to another one, as and when required.

Dibris

More details on JADE: The HalloWorld Agent

- A type of agent is created by extending the `jade.core.Agent` class and redefining the `setup()` method.
- Each Agent instance is identified by an AID (`jade.core.AID`).
  - An AID is composed of a unique name plus some addresses
  - An agent can retrieve its AID through the `getAID()` method of the `Agent` class

```
import jade.core.Agent;

public class HalloWorldAgent extends Agent {

  protected void setup() {
    System.out.println("Hallo World! my name is "+getAID().getName());
  }
}
```

Dibris

Local names, GUID and addresses

- Agent names are of the form <local-name>@<platform-name>
- The complete name of an agent must be globally unique.
- The default platform name is <main-host>:<main-port>/JADE
- The platform name can be set using the `-name` option
- Within a single JADE platform agents are referred through their names only.
- Given the name of an agent its AID can be created as
  - `AID id = new AID(localname, AID.ISLOCALNAME);`
  - `AID id = new AID(name, AID.ISGUID);`
- The addresses included in an AID are those of the platform MTPs and are ONLY used in communication between agents living on different FIPA platforms

Dibris

Passing arguments to an agent

- **It is possible to pass arguments to an agent**
  - `java jade.Boot .... a:myPackage.MyAgent(arg1 arg2)`
  - **The agent can retrieve its arguments through the `getArguments()` method of the `Agent` class**

```
protected void setup() {
  System.out.println("Hallo World! my name is "+getAID().getName());
  Object[] args = getArguments();
  if (args != null) {
    System.out.println("My arguments are:");
    for (int i = 0; i < args.length; ++i) {
      System.out.println("- "+args[i]);
    }
  }
}
```

Dibris

Agent termination

- **An agent terminates when its `doDelete()` method is called.**
- **On termination the agent's `takeDown()` method is invoked (intended to include clean-up operations).**

```
protected void setup() {
  System.out.println("Hallo World! my name is "+getAID().getName());
  Object[] args = getArguments();
  if (args != null) {
    System.out.println("My arguments are:");
    for (int i = 0; i < args.length; ++i) {
      System.out.println("- "+args[i]);
    }
  }
  doDelete();
}

protected void takeDown() {
  System.out.println("Bye...");
}
```

Dibris

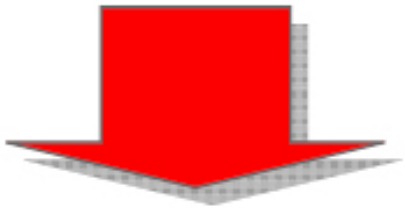
The Behaviour class

- The actual job that an agent does is typically carried out within "behaviours"
- Behaviours are created by extending the `jade.core.behaviours.Behaviour` class
- To make an agent execute a task it is sufficient to create an instance of the corresponding `Behaviour` subclass and call the `addBehaviour()` method of the `Agent` class.
- Each Behaviour subclass must implement
  - `public void action()`: what the behaviour actually does
  - `public boolean done()`: Whether the behaviour is finished

Dibris

Behaviour scheduling and execution

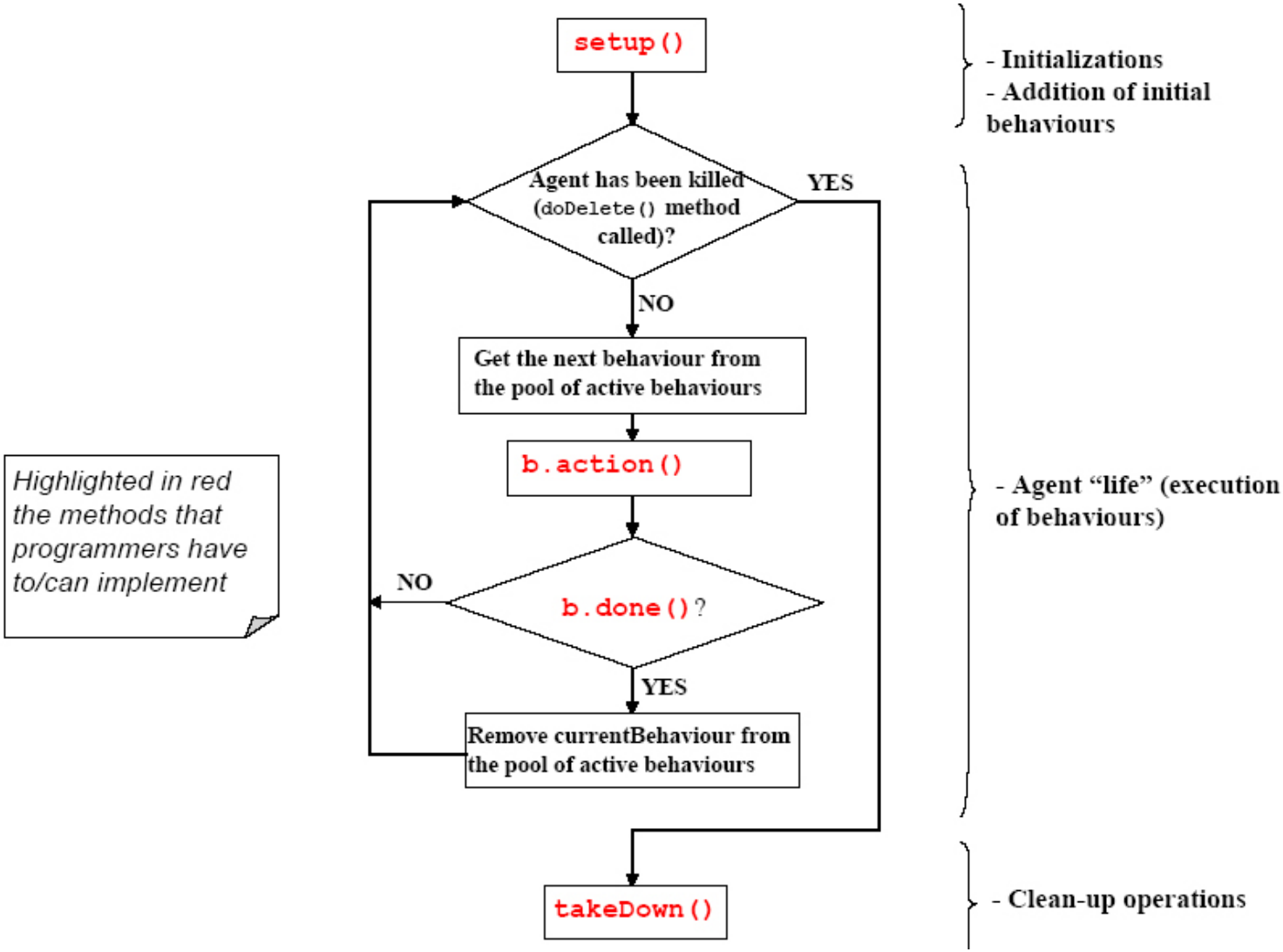
- An agent can execute several behaviours in parallel, however, behaviour scheduling is not preemptive, but cooperative and everything occurs within a single Java Thread



Behaviour switch occurs only when the `action()` method of the currently scheduled behaviour returns.

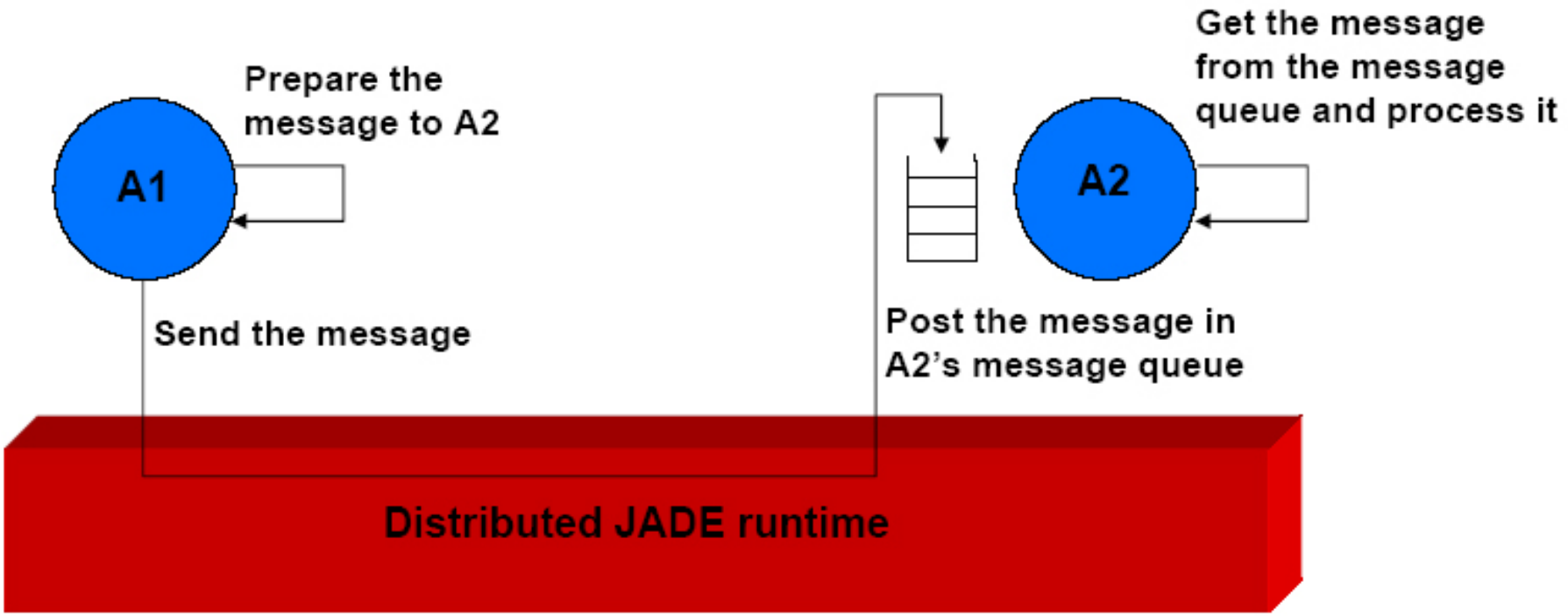
Dibris

The agent execution model



The communication model

- Based on asynchronous message passing
- Message format defined by the ACL language (FIPA)



The ACLMessage class

- **Messages exchanged by agents are instances of the `jade.lang.acl.ACLMessage` class.**
- **Provide accessor methods to get and set all the fields defined by the ACL language**
  - **get/setPerformative();**
  - **get/setSender();**
  - **add/getAllReceiver();**
  - **get/setLanguage();**
  - **get/setOntology();**
  - **get/setContent();**
  - **....**

Sending and receiving messages

- **Sending a message is as simple as creating an `ACLMessage` object and calling the `send()` method of the `Agent` class**

```
ACLMessage msg = new ACLMessage(ACLMessage.INFORM);
msg.addReceiver(new AID("Peter", AID.ISLOCALNAME));
msg.setLanguage("English");
msg.setOntology("Weather-Forecast-Ontology");
msg.setContent("Today it's raining");
send(msg);
```

- **Reading messages from the private message queue is accomplished through the `receive()` method of the `Agent` class.**

```
ACLMessage msg = receive();
if (msg != null) {
  // Process the message
}
```

Dibris

Blocking a behaviour waiting for a message

- A behaviour that processes incoming messages does not know exactly when a message will arrive → It should poll the message queue by continuously calling `myAgent.receive()`.
- This of course would completely waste the CPU time.
- The `block()` method of the `Behaviour` class removes a behaviour from the agent pool and puts it in a blocked state (it's not a blocking call!!).
- Each time a message is received all blocked behaviours are inserted back in the agent pool and have a chance to read and process the message.

```
public void action() {
  ACLMessage msg = myAgent.receive();
  if (msg != null) {
    // Process the message
  }
  else {
    block();
  }
}
```

This is the strongly recommended pattern to receive messages within a behaviour

Dibris

Selective reading from the message queue

- The `receive()` method returns the first message in the message queue and removes it.
- If there are two (or more) behaviours receiving messages, one may "steal" a message that the other one was interested in.
- To avoid this it is possible to read only messages with certain characteristics (e.g. whose sender is agent "Peter") specifying a `jade.lang.acl.MessageTemplate` parameter in the `receive()` method.

```
MessageTemplate tpl = MessageTemplate.MatchOntology("Test-Ontology");

public void action() {
  ACLMessage msg = myAgent.receive(tpl);
  if (msg != null) {
    // Process the message
  }
  else {
    block();
  }
}
```

Dibris

Receiving messages in blocking mode

- The `Agent` class also provides the `blockingReceive()` method that returns only when there is a message in the message queue.
- There are overloaded versions that accept a `MessageTemplate` (the method returns only when there is a message matching the template) and or a timeout (if it expires the method returns null).
- Since it is a blocking call it is "dangerous" to use `blockingReceive()` within a behaviour. In fact no other behaviour can run until `blockingReceive()` returns.

- Use `receive()` + `Behaviour.block()` to receive messages within behaviours.
- Use `blockingReceive()` to receive messages within the agent `setup()` and `takeDown()` methods.

Dibris

The yellow pages service

